GivingHand Project Overview

Project Description

GivingHand is a lightweight community-driven charity application designed to facilitate social impact and resource distribution in a collaborative environment. The application allows users to launch humanitarian campaigns, track real-time needs, and commit to life-changing donations. Users can register as donors or organizations, maintain impactful profiles, post urgent aid requests, and track the journey of their contributions from commitment to distribution.

The framework is built using a clean, modular Java EE architecture, utilizing EJBs for business logic, JPA for persistence, JTA for transaction management, JAX-RS for RESTful APIs, JAAS for role-based security, and JMS for asynchronous messaging and real-time donation notifications.

Cheating Policy

- Cheaters will be given a NEGATIVE grade and no excuses.
- Cheating means submitting a code you haven't written.
- "Not your Code" means: a friend code, a code generated from any AI tool, a code taken from a public github repo, or any other resource.

Project Deadline

- **Submission Date Phase 1**: Saturday, 18 April 11:55 PM
- **Submission Date Phase 2**: Saturday, 25 April 11:55 PM
- **This is a Strict Deadline that won't be changed.**

Project Organization and Team Collaboration

- **File Naming Convention**: Upload your project as a ZIP file with the format GroupNumber_ID1_ID2_ID3_ID4_TAName.zip.
- **Team Size**: Maximum of **3-4 students** per team **from the same lab or same slot**.
- The Project worths **150 points (15 Marks)**.
- **Collaboration**: Ensure equal distribution of workload among team members. Each member should understand the overall project and be able to work on any component.
- You must create a **postman collection** that includes all the API with ready request bodies and parameters. This collection will be evaluated at the discussion time. This will be a part of the grading criteria
- You need to implement your system **clean project architecture** to separate the project layers in different packages this point will be part of the grading criteria.

- **Note that: any violation in the assignment submission will cost you 0.5 mark of the assignment grade (like submitting more than one file, not following the naming convention, using different file extension)**

Core Features

1. User Management [10 Points] 📄

1.1 User Registration: Users can register for an account with a valid email address and password. During registration, users choose a specific role: **Donor** (Individual) or **Organization** (Charity).

- **Input:** Email, Password, Full Name, Birthday date, bio and Role (Donor/Organization).
- **Validation:** All fields must be filled except bio, the birthday and email should have extra validation.
- **Success/Error:** Returns a message on success or an error message with the specific failed field identified in the message.

1.2 User Login: Users authenticate to access the application.

1.3 Profile Management: Users can update their profile information (name and bio).

2. Campaign Management [30 Points] 📢

2.1 Create Campaigns: Organization users create humanitarian campaigns (e.g., "Emergency Flood Relief"). Each campaign includes a title, description, category and a target list of needed items (name and quantity).

2.2 Manage Active Needs: Organizations can update campaign status (Open, Paused, or Completed) and modify the list of required resources.

2.3 Discover Campaigns: All users can browse a directory of active campaigns (Open), filtering by category.

3. Donation & Commitment Tracking [40 Points] 🤝

3.1 Commit to Donate: Donors select specific items from a campaign's "Need List" and commit to providing them (e.g., 10 blankets).

3.2 Donation Lifecycle: The system tracks the journey through a defined workflow: **Committed → Received → Distributed**.

3.3 Edit/Cancel Commitment: Donors can update or cancel their commitment, ensuring the "Need List" reflects accurate remaining requirements. It checks first if the donation isn't received yet else no update can be made.

3.4 Donations List: Once a donation is marked as "Distributed," the Donor's profile is updated with a contribution history log.

4. Resource Tracking & Inventory [40 Points] 📦

4.1 Warehouse Setup: Organizations can define "Virtual Warehouses" to store their resources. Each warehouse tracks a list of inventory items (e.g., Medical Kits, Dry Food, Water Packs) and their current quantities.

4.2 Resource Allocation: Organizations can allocate items from their Warehouse to a specific Campaign.

- **Transaction Integrity:** the system ensures that the Warehouse stock decreases and the Campaign "Received" count increases simultaneously. If one step fails, the entire transaction rolls back.

4.3 Stock Monitoring (View Warehouse): The system provides a dashboard for Organizations to monitor their total available resources.

5. [Messaging Module] Notifications via JMS [15 Points] 📬

(sender/receiver)

- **Low Stock Alert:** Sent to an Organization when a critical resource falls below a **threshold** (warehouse).

  Donation Received: Sent to a Donor when their commitment reaches the organization's warehouse (when received).

- **Note:** Uses a standardized JSON event object containing task updates and event names. Implement messaging using Message-Driven Bean (MDB) and test them using console logs.

6. Authentication and Security [15 Points] 🔐

Role-Based Access Control (JAAS):

- **Organization Role:** Permissions to create campaigns, manage their own warehouses, and verify received donations.
- **Donor Role:** Access to browse campaigns and manage their own donation commitments.

API References (For testing purposes)

The following are some samples for testing some APIs to guide you of how the output should look like.

1 User Management APIs

1.1 Name: User Registration

URL: /api/register

Request Body:
JSON
```
{
  "email": "charity@helpinghands.org",
  "password": "SecurePassword123",
  "name": "Helping Hands Org",
  "bio": "Dedicated to disaster relief and food security.",
  "role": "organization" // Options: "donor" or "organization"
}
```

Response:
Status: 201 Created (If successful)
Status: 400 Bad Request (If validation fails)
Body:
JSON
```
{
  "message": "User registered successfully."
}
```

1.2 Name: User Login

URL: /api/login

Request Body:
JSON
```
{
  "email": "charity@helpinghands.org",
  "password": "SecurePassword123"
}
```

Response: Status: 200 OK (If successful)
Status: 401 Unauthorized (If credentials fail)
Body:

JSON
```
{
  "message": "Login successful.",
  "token": "JAAS-SESSION-TOKEN"
}
```

---

2 Campaign & Donation APIs

2.1 Name: Create Campaign

URL: /api/campaigns/create

Request Body:
JSON
```
{
  "title": "Winter Blanket Drive 2026",
  "description": "Collecting blankets for the upcoming cold wave.",
  "category": "Emergency Relief",
  "needed_items": [
    {"item_name": "Wool Blanket", "target_quantity": 100},
    {"item_name": "Heater", "target_quantity": 20}
  ]
}
```

Response: Status: 201 Created
Body:
JSON
```
{
  "message": "Campaign created successfully.",
  "campaign_id": 501
}
```

2.2 Name: Commit to Donate

URL: /api/donations/commit

Request Body:
JSON
```
{
  "campaign_id": 501,
  "item_name": "Wool Blanket",
  "quantity": 10
}
```

Response: Status: 200 OK

Body:
JSON
```
{
  "message": "Commitment recorded. Please drop off items at the warehouse.",
  "donation_id": 9001
}
```

---

3 Resource Tracking & Inventory APIs

3.1 Name: Add Warehouse Inventory

URL: /api/warehouse/{warehouse_id}/add

Request Body:
JSON
```
{
  "item_name": "First Aid Kit",
  "quantity": 50,
  "category": "Medical"
}
```

Response: Status: 200 OK
Body:
JSON
```
{
  "message": "Inventory updated successfully."
}
```

3.2 Name: Allocate Resources (JTA Protected)

URL: /api/inventory/allocate

Request Body:
JSON
```
{
  "warehouse_id": 10,
  "campaign_id": 501,
  "item_name": "First Aid Kit",
  "quantity": 20
}
```

Response: Status: 200 OK (If JTA transaction completes)
Status: 400 Bad Request (If insufficient stock)
Body:
JSON
```
{
```

"message": "Resources allocated from warehouse to campaign successfully."
}

---

4 Messaging & Notifications (JMS)

4.1 Name: Get Notifications

URL: /api/notifications
Response: Status: 200 OK
Body:
JSON

[
{
"event_type": "STOCK_LOW_ALERT",
"message": "Warning: First Aid Kit stock is below 10% in Warehouse 10.",
"timestamp": "2026-04-04T12:00:00Z"
},
{
"event_type": "DONATION_RECEIVED",
"message": "Your 10 Wool Blankets have been received by the organization.",
"timestamp": "2026-04-04T12:05:00Z"
}
]

Project Setup

Goal:

To introduce steps for setting up your local environment with the course tools to be able to deliver project.

Steps:

The environment setup can be configured with two methods:

a) Downloading the eclipse IDE and needed plugin

b) Downloading virtual box and import image attached

a) Method A:

1. download "eclipse ee: from next link
https://www.eclipse.org/downloads/packages/release/2021-03/r/eclipse-ide-enterprise-java-and-web-developers

2. add the "JBoss tools" plugin to your eclipse from this link
https://marketplace.eclipse.org/content/jboss-tools
You will just drag and drop the plugin into your eclipse and the installation will begin automatically

3. add "JBoss EAP server 7.0 or more" to your eclipse [download the zip file from the following link and add the home directory as we added it in the redhat environment]
https://drive.google.com/file/d/1MUXeSbO1H4JxvvQSMCk6fUSz8RcFS6M-/view?ts=626ab8a9

4. download postman
https://www.postman.com/downloads/

A video is uploaded to this link, shows the detailed steps:
https://drive.google.com/file/d/1T2uWsd4Yl7dgFELPCz5HxvD-S9z4uxsK/view?usp=sharing

b) Method B:

1. Download & install virtual box https://www.virtualbox.org/wiki/Downloads
2. Download & Import image
https://drive.google.com/file/d/1MFGUaczb6ACj1ZSN7_01U6dOGZwN-N-c/view?usp=sharing
3. Start the image
4. The image password is `rh_2020` and you can find eclipse in `\home\student\eclipse` and postman `\home\student\postman`
5. You can watch this video, for detailed steps
https://drive.google.com/file/d/1Z6Fh7QVMfI1akNpSeKtwIAtRkmm05aeZ/view?usp=sharing
6. Start watching the video from 1:30.
https://drive.google.com/file/d/1T2uWsd4Yl7dgFELPCz5HxvD-S9z4uxsK/view?usp=sharing